

ISAI (Variable Instruction Word Length)

Funktion	Opcode	Operand 1	Operand 2	Beschreibung
NOP	00			
Immediate	41	Value		Es landet auf dem Belt
Call Block	82	Block Nr.		Dieser Block kann direkt in den PC geladen werden. Danach wird keiner mehr zum Stack hinzugefügt.
Push Block	83	Block Nr.		Ein Block wird zum Stack hinzugefügt. Aber es ist nicht der letzte. Der PC muss auf den letzten Block warten.
Conditional Call Block	84	Block Nr.		
Conditional Push Block	85	Block Nr.		Der boolsche Wert wird geprüft. Wenn 1, dann findet der Push statt
A = 0	46	R		Gibt einen Boolschen Wert aus
A = 1	47	R		
A = B	88	R1	R2	
A > B	89	R1	R2	
If	0A			Setzt den boolschen Wert, der für die Push's geprüft wird, jenachdem wie der Entscheidungsbaum aussieht.
Elseif	0B			Erst die Logischen Funktionen. Danach kann man den Boolschen Wert mit If/Elseif/Else setzen (A=B? → IF → A=0? → ELSEIF → ELSE → ...)
Else	0C			
Renew	4D	R		Anstatt einen Wert dauerhaft im Register zu speichern, sind alle immer auf dem Belt. Braucht man einen Wert länger, legt man ihn einfach wieder an den Anfang, dass er nicht herunterfällt (Mehr Software und weniger Hardware)
Spill	8E	Reg Nr.	R	Wert R vom Belt ins Scratchpad schreiben
Fill	4F	Reg Nr.		Wert vom Scratchpad auf den Belt werfen
Pick Address (no Reset)	50	Addr. Nr.		Adresse wird im Memory Table nachgeschlagen und steht für Load/Store zur Verfügung.
Pick Address (Reset Offset)	51	Addr. Nr.		Adresse wird im Memory Table nachgeschlagen. Der Offset wird zurückgesetzt.
Load indirect Address	52			Es wird auf den Belt geladen von Adresse aus Memory Table + Offset
Store indirect Address	53			Belt Anfang wird auf Adresse von Memory Table geschrieben + Offset
Add Register indirect Offset	54	R		Zu der nachgeschlagenen Adresse kann ein Offset addiert werden. Dies kann auch mehrmals geschehen. So kann ein beliebig großer Address-Raum durchsucht werden (ohne noch mehr Operanden oder Instructions)
Sub Register Indirect Offset	55	R		Jede Adresse hat ihren eigenen Offset. Dieser kann individuell zurückgesetzt werden.
Add	96	R1	R2	Es gibt kein Ziel Register, da das Ergebnis auf einem Belt landet. Es muss ggf. manuell ins Register kopiert werden. Nach Add muss für das Ergebnis 2 Schritte gewartet werden
Subtract	97	R1	R2	
Multiply	98	R1	R2	
Bit Shift	59	R		Verschiebe die Bits um 1
Increment	5A	R		Addiere 1
Decrement	5B	R		
Not	5C	R		
And	9D	R1	R2	
Or	9E	R1	R2	

Info: Es gibt kein Halt. Wenn es keinen auszuführenden Codeblock mehr gibt, steht alles still.

Interrupt ist auch nicht nötig. Die Routine wird einfach in den Stack geschoben.

Idee: Memory Space Map

Idee: Gerber mit Python

Die ersten zwei Bits sind für die Anzahl der Operanden. Die weiteren eine fortlaufende Zahl.